

# The Project Explained from Scratch

Explainable network intrusion detection with LLM-written incident reports, in plain English

## The one-sentence version

**We're building a system that watches computer network traffic, spots the traffic that looks like a cyberattack, shows *why* it looked suspicious, and then writes a short plain-English report telling a human what's happening, how serious it is, and what to do about it.**

## The world this lives in

Every organisation, whether a bank, hospital, university or online shop, has computers talking to each other and to the internet all day. Each conversation is called a **connection**, and a busy company can have millions of them an hour. Most are harmless, like someone opening a web page or sending an email. A few are attacks: someone trying to break in, knock the service offline, or steal something.

Large organisations employ a team called a **Security Operations Center (SOC)**. Think of it as the control room of a building's security guards, watching screens and responding when something looks wrong. The people in it are called **analysts**.

## The problem

Three things go wrong in that control room.

### 1. There's far too much to watch

No human can read millions of connections. Software has to pre-filter them, and analysts only look at what the software flags. If the software misses an attack, nobody notices until the damage is done.

### 2. The software can't explain itself

Modern machine-learning models can be very accurate, but they behave like a black box. They say “this is an attack, 93% sure” and nothing more. An analyst can't act on that. They need to know *why*, because they have to decide whether to trust it, investigate, or ignore it. A guard who says “trust me, that man is dangerous” without saying why isn't much use.

### 3. Turning an alert into action is slow, manual work

For each alert, an analyst has to work out what it means, write it up as a ticket, judge how urgent it is, and decide what to do. Doing this for hundreds of alerts a day is exhausting, and tired people make mistakes.

## What we're trying to do

We're building a pipeline that automates the first pass of that work, like a junior analyst who prepares the paperwork so the senior human can make the final call. It has four stages.

## Stage 1: Prepare the data

Raw network records are messy, and computers can only do maths on numbers. We clean the data, turn words like “tcp” or “http” into numbers, and put all measurements on a comparable scale. Think of it as translating a pile of handwritten notes into one tidy spreadsheet.

## Stage 2: Detect and classify

We train a machine-learning model on thousands of past connections that have already been labelled “normal” or “this type of attack.” It learns the patterns and can then label new, unseen connections. We train three different models (a simple one, a more powerful one, and a small neural network) and pick the best, because we want to understand how they differ.

## Stage 3: Explain the decision

We use a technique called **SHAP** to show which pieces of evidence pushed the model toward its verdict. Imagine a team project where you want to know how much each member contributed to the final grade. SHAP does that for the model's clues, saying things like “the huge number of connections in two seconds contributed most, and the high error rate was second.”

## Stage 4: Write the report

We feed the verdict and its evidence to a **Large Language Model (LLM)**, the same kind of technology behind chatbots like ChatGPT or Claude. It turns the technical evidence into a short plain-English note covering what happened, how severe it is, and what the analyst should do. We run the LLM on our own laptops using free software called Ollama, so it costs nothing.

## The most important design rule

**The LLM is the narrator, not the detector. It never decides whether something is an attack. The trained classifier does that. The LLM only explains a decision that has already been made, using only the evidence we hand it.**

We do this because LLMs sometimes “hallucinate,” meaning they confidently make things up. If we let one judge attacks, it might invent threats or miss real ones, and it would sound convincing either way. By giving it a narrow job with controlled inputs, and by automatically checking that its report only mentions facts we gave it, we keep the risk small.

## What the data looks like

We use a well-known practice dataset called **NSL-KDD**. It's a collection of network connection records, each described by 41 measurements (how long it lasted, how much data was sent, how many failed login attempts occurred, and so on), plus a label saying what it was. Every record falls into one of five categories:

- **Normal:** ordinary, harmless traffic.
- **DoS (Denial of Service):** flooding a service with so many requests that real users can't get in. It's like a mob crowding a shop's entrance so customers can't enter.
- **Probe:** an attacker scouting for weaknesses, such as scanning which doors and windows are unlocked before breaking in.

- **R2L (Remote to Local):** an outsider with no account tricking their way in over the network, for example by guessing passwords.
- **U2R (User to Root):** someone who already has a normal account gaining administrator-level control, like a visitor with a day pass finding a way to the master key.

## Why the project is harder than it sounds

### Rare attacks are easy to miss

The training data has about 67,000 normal records and 46,000 DoS records, but only 52 examples of the U2R attack. A lazy model could ignore the rare attacks and still score high overall, because most traffic is normal. So we measure success by how well we catch *each* type, especially the rare ones, not by overall accuracy. If the model does badly on U2R, we report that honestly as a finding.

### Keeping the test fair

If a student sees the exam questions beforehand, their score means nothing. Similarly, we keep the test data completely separate while preparing and training the model. Otherwise our results would look better than they really are.

### Explanations can mislead

SHAP shows what the *model* relied on, which isn't always what *actually caused* the attack. We check that the model's reasoning makes sense, for example that DoS alerts are driven by connection-count features.

## The experiment at the end

In Week 4 we run a side experiment. We take a set of connections the model has never seen and ask the LLM to classify them on its own, with no trained classifier, just the raw measurements. Then we compare the LLM's accuracy against our trained model.

The point is to test whether a clever language model can replace a purpose-built detector. We expect the trained model to win clearly, especially since our LLM is a small one running on a laptop. That's a useful result. It supports the design choice of letting the specialist detect and the LLM only explain.

## A worked example

*This is an invented illustration of the finished system, not real output.*

A connection record arrives. The classifier says **DoS, 96% confident**. SHAP says the top clues were that the same machine opened 511 connections to the server in two seconds, and nearly all of them failed. The LLM then writes something like:

**Summary:** One source machine sent 511 connection attempts to the same server within two seconds, and almost all failed. This pattern matches a denial-of-service attack trying to overwhelm the server.

**Severity:** High.

**Recommended action:** Block the source address at the firewall and check whether the affected service is still responding normally.

Our automatic checker then confirms the report only cites evidence that was in the input. A human analyst glances at it and acts within seconds, instead of piecing it together from scratch.

## How two people split the work

To avoid waiting on each other, one of you builds the data, models and explanations, and the other builds the LLM report-writer. The thing that lets you work separately is a **contract**, an agreed format for what the first half hands to the second. It's like two people building opposite halves of a bridge who agree up front exactly where the halves meet. Early on, the data person provides fake sample hand-offs so the LLM person can start immediately.

## What this project is not

- It's not a finished security product. It's a learning project on a practice dataset.
- NSL-KDD is old, so real modern attacks look different. It's a good learning tool, not a perfect picture of today's threats.
- The “severity” rating is judged against our own scoring guide, because there's no official correct answer.
- The LLM can still make mistakes, which is why we check its work and review samples by hand.
- Because NSL-KDD is public, the LLM may have seen it during its training. We mention that as a limitation.

## What success looks like

By the end you'll have a working pipeline that takes a network connection and produces a verdict, the evidence behind it, and a readable incident report. You'll also have honest numbers on how well each model performed, including where they fail, and a written README on GitHub explaining the method, results and limitations. Because you build every stage yourselves, you'll understand what happens inside each one, which was the goal.